

PROJECT REPORT

AI HR Recruitment Assistant

Using Agentic AI and Retrieval-Augmented Generation (RAG)

IBM Internship Project

Prepared based on the supplied IBM internship project template and project specification.

1. Project Overview

1.1 Project Title

AI HR Recruitment Assistant Using Agentic AI and Retrieval-Augmented Generation

1.2 Problem Statement

Recruitment teams often need to process large numbers of resumes for a single job opening. Manually reviewing resumes, comparing candidate qualifications with job requirements, identifying skill gaps, ranking candidates, and preparing interview questions can be time-consuming and inconsistent.

Traditional resume-management systems mainly store candidate information and provide keyword-based search. They do not provide intelligent reasoning about why a candidate is suitable for a particular role or provide evidence supporting the recommendation.

The proposed project addresses this problem by developing an intelligent AI HR Recruitment Assistant that combines an AI Agent, specialized tools, Retrieval-Augmented Generation (RAG), vector search, structured candidate data, and explainable candidate matching.

1.3 Brief Description of the Project

The AI HR Recruitment Assistant is an enterprise-style recruitment workspace designed to assist recruiters throughout the candidate-screening process.

Recruiters can create or upload Job Descriptions and upload one or multiple resumes. The system processes the documents, extracts structured candidate information, generates embeddings, and stores resume information in a searchable knowledge base.

The Recruitment AI Agent can understand recruiter requests and determine which tools and information are required. It can perform candidate searches, retrieve semantic resume evidence, match candidates with jobs, calculate explainable match scores, rank candidates, identify skill gaps, compare candidates, and generate personalized interview questions.

2. Objectives & Proposed Solution

2.1 Project Objectives

- Develop an intelligent AI-powered recruitment assistant.
- Automate resume analysis and candidate information extraction.
- Analyze Job Descriptions and convert them into structured requirements.
- Create a searchable candidate knowledge base using RAG and vector embeddings.
- Match candidates against job requirements using transparent scoring.
- Rank candidates according to job-related qualifications.
- Provide explainable AI recommendations supported by resume evidence.
- Identify candidate skill gaps and compare multiple candidates.
- Generate personalized interview questions and interview plans.
- Provide recruiters with a natural-language AI recruitment assistant.
- Implement secure backend APIs and role-based authorization.
- Support Demo Mode using realistic deterministic data when external AI services are unavailable.

2.2 Proposed Agentic AI Solution

The central component of the system is the Recruitment AI Agent. Instead of using one large AI prompt, the agent determines the user's intent and selects the appropriate backend tools.

For example, when a recruiter asks, "Find the best candidates for this Senior React Developer job," the agent can identify the selected job, retrieve structured requirements, search candidates, perform semantic RAG retrieval, match candidates, calculate scores, rank candidates, and return an explainable result.

2.3 Agent + Tools + RAG Architecture

Agent Layer: Receives the recruiter's natural-language request and determines what actions are required.

Tools Layer: Provides specialized capabilities instead of allowing the agent to directly perform arbitrary database operations.

- Resume Parser Tool
- Job Description Analyzer Tool
- Candidate Search Tool
- Candidate Retrieval Tool
- RAG Search Tool
- Candidate Matching Tool
- Candidate Ranking Tool
- Skill Gap Analysis Tool
- Interview Question Generator Tool
- Candidate Database Tool
- Candidate Comparison Tool

RAG Layer: Uploaded resumes are converted into text, cleaned, chunked, embedded, indexed, and semantically retrieved when evidence is required.

3. Key Features

3.1 Resume Processing

Supports bulk PDF, DOC, and DOCX resume processing with stages including Uploading, Parsing, Extracting Information, Generating Embeddings, Indexing, and Completed.

3.2 Job Description Analysis

Extracts required and preferred skills, experience, responsibilities, education, certifications, domain experience, technologies, and seniority.

3.3 Explainable Candidate Matching

Calculates a 0–100 score and explains matched skills, missing skills, relevant experience, projects, domain matches, gaps, and supporting resume evidence.

3.4 Candidate Ranking

Ranks candidates from strongest to weakest match and provides search, filtering, sorting, pagination, and recruitment-status controls.

3.5 Candidate Profile

Provides an AI-generated summary, score breakdown, skills, experience, education, certifications, projects, strengths, gaps, resume evidence, notes, status, and interview questions.

3.6 Personalized Interview Generation

Generates Technical, Behavioral, Situational, Resume-Based, and Role-Specific questions using both candidate and job information.

3.7 AI Interview Planner

Creates an editable interview plan based on candidate, job, interview type, and duration.

3.8 AI Recruitment Chat Assistant

Allows recruiters to ask natural-language questions about candidates and jobs through the backend Recruitment Agent.

4. Implementation & Results

4.1 Technologies and Tools Used

- **Frontend:** React, TypeScript, Ant Design, Framer Motion or equivalent, Recharts, and custom CSS.
- **Backend:** TypeScript, REST APIs, Recruitment Agent architecture, specialized tools, and Zod validation.
- **Database:** PostgreSQL and pgvector or an equivalent vector database.
- **AI/RAG:** Large Language Model integration, embedding generation, vector similarity search, Retrieval-Augmented Generation, and structured AI outputs.

4.2 Candidate Matching Score

Matching Factor	Weight
Required Skills	35%
Experience	20%
Responsibilities	15%
Domain Experience	10%
Preferred Skills	10%
Education	5%
Certifications	5%
Total	100%

4.3 Database Design

The proposed PostgreSQL schema includes users, jobs, candidates, resumes, candidate skills, candidate experience, candidate education, candidate projects, job requirements, candidate matches, interview questions, recruiter notes, documents, and document chunks.

Foreign keys, indexes, timestamps, and constraints maintain data integrity. Vector embeddings are associated with document chunks to support semantic candidate retrieval.

4.4 Working Process

- **Step 1:** Recruiter opens the dashboard.
- **Step 2:** Recruiter creates a job or uploads a Job Description.
- **Step 3:** The Job Description Analyzer extracts structured requirements.
- **Step 4:** Recruiter uploads one or multiple resumes.
- **Step 5:** The system parses and extracts candidate information.
- **Step 6:** Resume content is chunked and converted into vector embeddings.
- **Step 7:** Structured data is stored in PostgreSQL and resume chunks are indexed for semantic retrieval.
- **Step 8:** Recruiter selects a job and chooses Find Best Candidates.
- **Step 9:** The Recruitment Agent determines the necessary tools and RAG operations.
- **Step 10:** The matching engine calculates transparent scores.
- **Step 11:** Candidates are ranked from strongest to weakest match.
- **Step 12:** The recruiter reviews match explanations and resume evidence.
- **Step 13:** The recruiter opens a candidate profile.
- **Step 14:** The system generates personalized interview questions.

4.5 Application Modules

- **Dashboard:** Recruitment statistics, pipeline, hiring funnel, match-score distribution, skills, job activity, and AI Assistant.
- **Candidates:** Candidate table/card view, search, filters, sorting, pagination, comparison, and status management.
- **Jobs:** Job creation, editing, JD analysis, requirements, candidate rankings, and interview planning.
- **Resume Screening:** Bulk upload and visible document-processing progress.
- **Candidate Matching:** Job selection, best-candidate retrieval, match scores, and match analysis.
- **Analytics:** Candidate screening, match scores, skills, pipeline, applicant volume, and recruitment conversion.
- **Settings:** AI configuration, matching weights, recruiter preferences, roles, and application configuration.

5. Security and Responsible AI

The system supports role-based authorization for ADMIN, RECRUITER, HIRING_MANAGER, and INTERVIEWER roles.

Security mechanisms include API validation, secure file uploads, file-size limits, MIME-type validation, rate limiting, input sanitization, audit logging, and secure environment-variable management.

Protected or sensitive characteristics must not be used as candidate-ranking criteria. The system should not infer race, religion, gender, age, disability, sexual orientation, political affiliation, or other protected characteristics from resumes.

Candidate recommendations are decision-support tools. Final hiring decisions should remain with qualified human professionals.

6. Demo Mode

Demo Mode allows the system to remain usable when PostgreSQL, an embedding provider, or an external LLM API is unavailable.

- At least 10 realistic sample candidates.
- 3 Job Descriptions.
- Candidate skills and experience information.
- Match scores and match explanations.
- Resume evidence.
- Interview questions.

Interfaces are used so mock AI services can later be replaced by real LLM and embedding providers without requiring major frontend changes.

7. Expected Results / Project Outcomes

- Reduction in manual resume-screening effort.
- Faster identification of suitable candidates.
- Semantic search across candidate resumes.
- Explainable candidate matching instead of unexplained ranking.
- Automated identification of skill gaps.
- Candidate comparison through structured AI analysis.
- Personalized interview-question generation.
- A centralized recruiter workspace.
- Consistent candidate evaluation based on job-related factors.
- Improved recruiter productivity through AI-assisted workflows.

8. Challenges Faced

Resume Data Extraction

Resumes have different layouts and formats, making reliable extraction of skills, experience, projects, and employment relationships challenging.

Semantic Retrieval

Keyword search alone is insufficient, requiring embeddings and vector retrieval to understand semantic relationships.

Explainability

AI match scores need supporting evidence, so structured candidate information must be combined with retrieved resume chunks.

AI Hallucination

Recruitment requires reliable facts; responses should therefore be grounded in database records and resume evidence.

Structured AI Responses

Zod validation and structured outputs are required to ensure consistent data formats.

System Reliability

LLM calls, embedding generation, parsing, database operations, and vector retrieval can fail, requiring graceful error handling and fallback behavior.

9. Conclusion

The AI HR Recruitment Assistant demonstrates how Agentic AI and Retrieval-Augmented Generation can be applied to modern recruitment workflows.

Unlike a traditional resume-management application, the system combines a Recruitment AI Agent with specialized tools, structured databases, semantic vector retrieval, explainable candidate matching, ranking, skill-gap analysis, candidate comparison, and personalized interview generation.

The architecture separates the React frontend from backend AI intelligence. The frontend presents information and communicates through APIs, while the backend manages the Agent, Tools, RAG pipeline, database operations, matching engine, and AI services.

This architecture provides a foundation that can be extended from a demonstration system into a scalable recruitment technology platform.

10. Future Scope

- Integration with newer and more capable LLMs.
- Multi-agent architecture with specialized Resume, Matching, Interview, Analytics, and Hiring agents.
- Advanced recruitment analytics and hiring-funnel optimization.
- Integration with Applicant Tracking Systems and HR Management Systems.
- Calendar, email, and enterprise identity integrations.
- Scalable vector search for thousands or millions of candidate records.
- Interview feedback analysis.
- Continuous improvement using recruiter feedback while maintaining responsible AI constraints.
- Advanced evidence tracing for individual match-score components.

11. References

1. IBM Internship Project Template – Project structure covering Project Overview, Objectives & Proposed Solution, Implementation & Results, and Conclusion & Future Scope.
2. AI HR Recruitment Assistant Project Specification – Agent + Tools + RAG architecture, recruitment workflow, database design, UI requirements, security requirements, and Demo Mode.
3. AI HR Recruitment Assistant Project Specification – Frontend, backend, database, API, security, testing, and deployment requirements.